

EEE373L

Embedded System Design Laboratory

Lab Report



Inspiring Excellence

Section:02

Group No:02

Experiment no:05

Name of the experiment: **Development of a Stopwatch (Use of Timer1)**

Prepared by:

Name: Yasir Anaf Siddique

ID: 21221073

Signature

All Group members:

Sl.	Name	ID	Signature
1.	Afrida Islam	21221030	
2.	Md Talha Bin Imran	21221018	
3.	Md. Hamin-Al-Araf	21221019	
4.	Nafim Karim Khan	21221039	

Date of Submission: 22/09/2024

Objective:

The objective of this experiment is to develop a stopwatch for measuring time in seconds and minutes. The stopwatch should have starting, stopping and resetting facilities.

Apparatus:

1. ATmega32
2. 7 Segment Display
3. Connecting Wires

Connection Diagram:

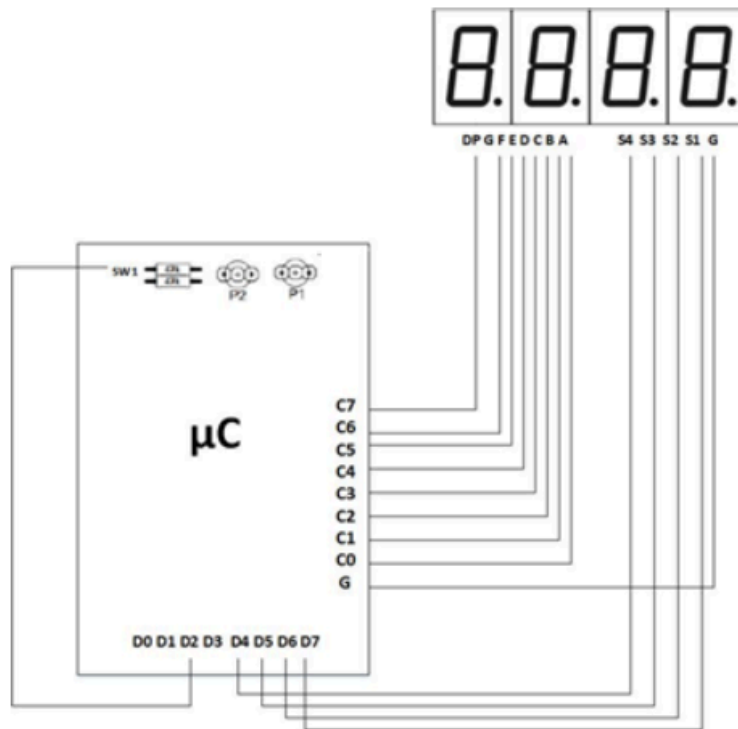


Figure-1: Circuit Diagram of a Stopwatch using ATmega32

The Code:

```
#include <mega32.h>
#include <delay.h>

// Define ports for 7-segment display
#define output_port PORTC
#define control_port PORTD

// Variables to hold the current time (seconds and minutes)
char sec = 45, min = 57, s1, s2, m1, m2, press = 0;

// Array showing the 7-segment cathode patterns for digits 0-9
char digit_cathod[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

char digit_cathod_decimal[10] = {0xBF, 0x86, 0xDB, 0xCF, 0xE6, 0xED, 0xFD, 0x87, 0xFF, 0xEF}; // | 0x80

// Function to separate the digits of seconds and minutes
void seperate_digits(void)
{
    s1 = sec % 10;
    s2 = sec / 10;
    m1 = min % 10;
    m2 = min / 10;
}

// Function to display the digits on the 7-segment display
void display_digits(void)
{
    control_port = 0b00010000;
    output_port = digit_cathod[s1];
    delay_ms(10);

    control_port = 0b00100000;
    output_port = digit_cathod[s2];
    delay_ms(10);

    control_port = 0b01000000;
    output_port = digit_cathod_decimal[m1];
    delay_ms(10);

    control_port = 0b10000000;
    output_port = digit_cathod[m2];
    delay_ms(10);
}
```

```

// Interrupt service routine for external interrupt INT0
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    press++;                // Increment the button press counter
    if (press == 3)         // Reset time after 3 presses
    {
        press = 0;
        min = 0;
        sec = 0;
    }
}

// Interrupt service routine for Timer1 Compare A
interrupt [TIM1_COMPA] void timer1_compa_isr(void)
{
    // Reset the Timer1 counter
    TCNT1H = 0x00;
    TCNT1L = 0x00;

    if (press == 1)
    {
        sec++;              // Increment seconds
        if (sec == 60)      // If seconds reach 60, reset to 0 and increment
            minutes
            {
                sec = 0;
                min++;
                if (min == 60) // If minutes reach 60, reset to 0
                    min = 0;
            }
    }
}

void main(void)
{
    DDRC = 0xFF; // Set PORTC as output (for the 7-segment display)

    DDRD = 0xF0; // Set the upper 4 bits of PORTD as output (for controlling the
7-segment displays)

    // Configure Timer1 for normal mode with a 1024 prescaler
    TCCR1A = (0 << COM1A1) | (0 << COM1A0) | (0 << COM1B1) | (0 << COM1B0) | (0 <<
WGM11) | (0 << WGM10);
    TCCR1B = (0 << ICNC1) | (0 << ICES1) | (0 << WGM13) | (0 << WGM12) | (1 << CS12) |
(0 << CS11) | (1 << CS10);

    TCNT1H = 0x00;
    TCNT1L = 0x00;
}

```

```

// Set the Timer1 Compare A register to a specific value
OCR1AH = 0x3D;
OCR1AL = 0x09;

TIMSK = (0 << OCIE2) | (0 << TOIE2) | (0 << TICIE1) | (1 << OCIE1A) | (0 << OCIE1B)
| (0 << TOIE1) | (0 << OCIE0) | (0 << TOIE0); // Enable Timer1 Compare A interrupt

GICR |= (0 << INT1) | (1 << INT0) | (0 << INT2); // Enable external interrupt INT0

MCUCR = (0 << ISC11) | (0 << ISC10) | (1 << ISC01) | (0 << ISC00); // Set INT0 to
trigger on a falling edge

GIFR = (0 << INTF1) | (1 << INTF0) | (0 << INTF2); // Clear INT0 flag

#asm("sei") // Enable global interrupts

// loop to update and display the time continuously
while (1)
{
    seperate_digits();
    display_digits();
}
}

```

Outputs:

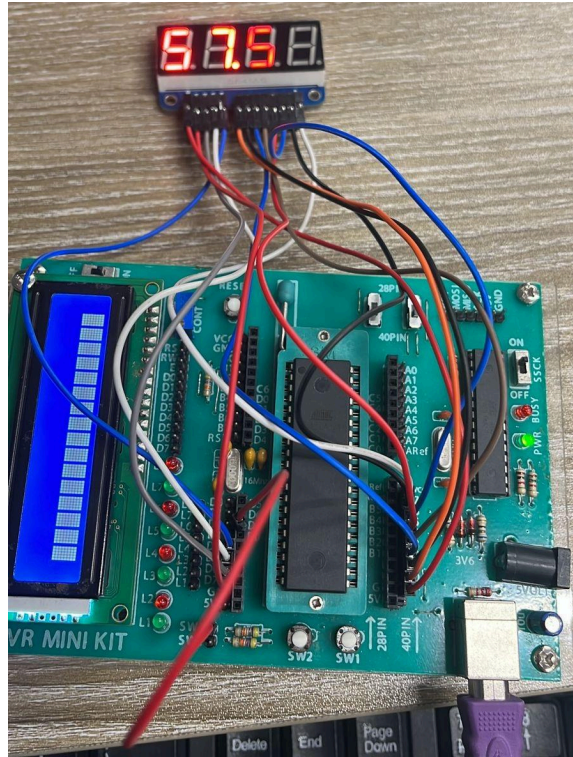


Figure-2: Timer1 digital Stopwatch in running state

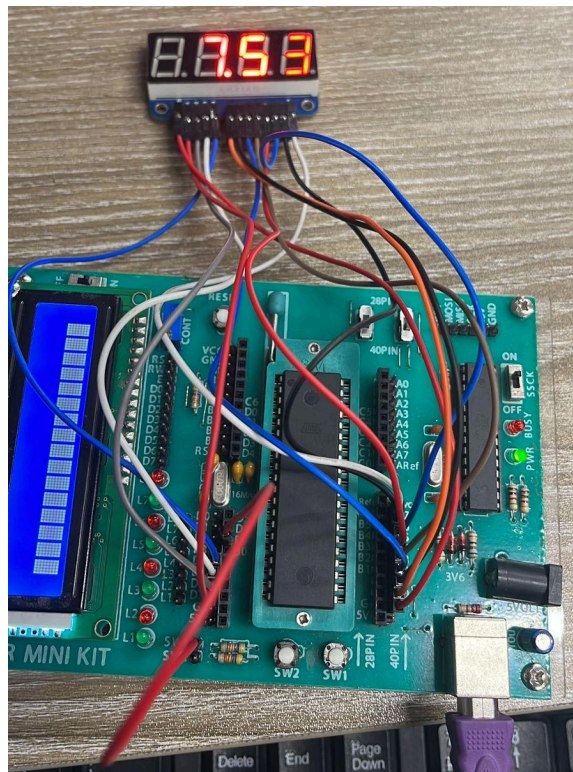


Figure-3: Timer1 digital Stopwatch in stop/ pause state

Discussion:

In this experiment, a stopwatch was implemented using the ATmega32 microcontroller, 16-bit Timer1, and 7-segment displays. Timer1 was configured with a prescaler of 1024, and in output compare match mode, it reached the value 0x3D09 to increment the time every second. The time was displayed on four 7-segment displays, showing minutes and seconds, using the functions `seperate_digits()` and `display_digits()`. External interrupts were used to pause and reset the timer, triggered by an external button press. When the button was pressed once, the time paused, and when pressed again, the timer reset to 0. Additionally, multiplexing was employed to efficiently manage the four 7-segment displays, while the Timer1 interrupt service routine ensured that the time was updated accurately every second.

Lab Report Tasks:

1. Code-

```
#include <mega32.h>
#include <delay.h>

// Define ports for 7-segment display
#define output_port PORTC
#define control_port PORTD

// Variables to hold the current time (seconds and minutes)
char sec = 45, min = 57, s1, s2, m1, m2, press = 0;
char pause = 0; // Flag to indicate pause state

// Array showing the 7-segment cathode patterns for digits 0-9
char digit_cathod[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

char digit_cathod_decimal[10] = {0xBF, 0x86, 0xDB, 0xCF, 0xE6, 0xED, 0xFD, 0x87, 0xFF, 0xEF}; // | 0x80

// Function to separate the digits of seconds and minutes
void seperate_digits(void)
{
    s1 = sec % 10;
    s2 = sec / 10;
    m1 = min % 10;
    m2 = min / 10;
}

// Function to display the digits on the 7-segment display
void display_digits(void)
{
    // If paused, blink the display
    if (pause == 1)
```

```

{
    // Turn off the display for blinking effect
    output_port = 0x00;
    delay_ms(300); // Adjust blink speed here
    return;
}

control_port = 0b00010000;
output_port = digit_cathod[s1];
delay_ms(10);

control_port = 0b00100000;
output_port = digit_cathod[s2];
delay_ms(10);

control_port = 0b01000000;
output_port = digit_cathod_decimal[m1];
delay_ms(10);

control_port = 0b10000000;
output_port = digit_cathod[m2];
delay_ms(10);
}

// Interrupt service routine for external interrupt INT0
interrupt [EXT_INT0] void ext_int0_isr(void)
{
    press++; // Increment the button press counter
    if (press == 3) // Reset time after 3 presses
    {
        press = 0;
        min = 0;
        sec = 0;
    }
    else if (press == 1) // Enter pause state
    {
        pause = 1; // Set pause flag to enable blinking
    }
    else if (press == 2) // Exit pause state
    {
        pause = 0; // Clear pause flag to stop blinking
    }
}

// Interrupt service routine for Timer1 Compare A
interrupt [TIM1_COMPA] void timer1_compa_isr(void)
{
    // Reset the Timer1 counter

```

```

    TCNT1H = 0x00;
    TCNT1L = 0x00;

    if (press == 1)
    {
        sec++; // Increment seconds
        if (sec == 60) // If seconds reach 60, reset to 0 and increment
minutes
        {
            sec = 0;
            min++;
            if (min == 60) // If minutes reach 60, reset to 0
                min = 0;
        }
    }
}

void main(void)
{
    DDRC = 0xFF; // Set PORTC as output (for the 7-segment display)

    DDRD = 0xF0; // Set the upper 4 bits of PORTD as output (for controlling the
7-segment displays)

    // Configure Timer1 for normal mode with a 1024 prescaler
    TCCR1A = (0 << COM1A1) | (0 << COM1A0) | (0 << COM1B1) | (0 << COM1B0) | (0 <<
WGM11) | (0 << WGM10);
    TCCR1B = (0 << ICNC1) | (0 << ICES1) | (0 << WGM13) | (0 << WGM12) | (1 << CS12) |
(0 << CS11) | (1 << CS10);

    TCNT1H = 0x00;
    TCNT1L = 0x00;

    // Set the Timer1 Compare A register to a specific value
    OCR1AH = 0x3D;
    OCR1AL = 0x09;

    TIMSK = (0 << OCIE2) | (0 << TOIE2) | (0 << TICIE1) | (1 << OCIE1A) | (0 << OCIE1B)
| (0 << TOIE1) | (0 << OCIE0) | (0 << TOIE0); // Enable Timer1 Compare A interrupt

    GICR |= (0 << INT1) | (1 << INT0) | (0 << INT2); // Enable external interrupt INT0

    MCUCR = (0 << ISC11) | (0 << ISC10) | (1 << ISC01) | (0 << ISC00); // Set INT0 to
trigger on a falling edge

    GIFR = (0 << INTF1) | (1 << INTF0) | (0 << INTF2); // Clear INT0 flag

    #asm("sei") // Enable global interrupts

```

```

// loop to update and display the time continuously
while (1)
{
    seperate_digits();
    display_digits();
}
}

```

Discussion: In the updated code, a pause flag is added which will be used to track when the stopwatch is paused. This detection will allow the display to blink during this pause state. When the stopwatch is paused, the `display_digits()` function checks whether this flag is set. If detected, it turns off and turns on the 7-segment display, creating the blink effect.

2. Code-

```

#include <mega32.h>
#include <delay.h>

#define output_port PORTC
#define control_port PORTD

char sec = 45, min = 57, hour = 1; // Added hour variable
char s1, s2, m1, m2, h1, h2, press = 0;
char digit_cathod[10] = {0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};
char digit_cathod_decimal[10] = {0xBF, 0x86, 0xDB, 0xCF, 0xE6, 0xED, 0xFD, 0x87, 0xFF, 0xEF};

void separete_digits(void)
{
    s1 = sec % 10;
    s2 = sec / 10;
    m1 = min % 10;
    m2 = min / 10;
    h1 = hour % 10; // Separate hour digits
    h2 = hour / 10;
}

void display_digits(void)
{
    control_port = 0b00000001; // Display first digit of hours

```

```

output_port = digit_cathod[h2];
delay_ms(10);

control_port = 0b00000010; // Display second digit of hours
output_port = digit_cathod[h1];
delay_ms(10);

control_port = 0b00000100; // Display first digit of minutes
output_port = digit_cathod[m2];
delay_ms(10);

control_port = 0b00001000; // Display second digit of minutes
output_port = digit_cathod_decimal[m1]; // With decimal for clarity
delay_ms(10);

control_port = 0b00010000; // Display first digit of seconds
output_port = digit_cathod[s2];
delay_ms(10);

control_port = 0b00100000; // Display second digit of seconds
output_port = digit_cathod[s1];
delay_ms(10);
}

interrupt [EXT_INT0] void ext_int0_isr(void)
{
    press++;
    if (press == 3)
    {
        press = 0;
        hour = 0;
        min = 0;
        sec = 0;
    }
}

interrupt [TIM1_COMPA] void timer1_compa_isr(void)
{
    TCNT1H = 0x00;
    TCNT1L = 0x00;
    if (press == 1)
    {
        sec++;
        if (sec == 60)
        {
            sec = 0;
            min++;
            if (min == 60)

```

```

        {
            min = 0;
            hour++; // Increment hour when minute reaches 60
            if (hour == 24) hour = 0; // Reset hours after 24
        }
    }
}

void main(void)
{
    DDRC = 0xFF; // Set PORTC as output for 7-segment display
    DDRD = 0x3F; // Set PORTD as output for control signals

    TCCR1A = (0 << COM1A1) | (0 << COM1A0) | (0 << COM1B1) | (0 << COM1B0) | (0 <<
WGM11) | (0 << WGM10);
    TCCR1B = (0 << ICNC1) | (0 << ICES1) | (0 << WGM13) | (0 << WGM12) | (1 << CS12) |
(0 << CS11) | (1 << CS10);
    TCNT1H = 0x00;
    TCNT1L = 0x00;
    ICR1H = 0x00;
    ICR1L = 0x00;
    OCR1AH = 0x3D;
    OCR1AL = 0x09;
    OCR1BH = 0x00;
    OCR1BL = 0x00;

    TIMSK = (0 << OCIE2) | (0 << TOIE2) | (0 << TICIE1) | (1 << OCIE1A) | (0 << OCIE1B)
| (0 << TOIE1) | (0 << OCIE0) | (0 << TOIE0);

    GICR |= (0 << INT1) | (1 << INT0) | (0 << INT2);
    MCUCR = (0 << ISC11) | (0 << ISC10) | (1 << ISC01) | (0 << ISC00);
    MCUCSR = (0 << ISC2);
    GIFR = (0 << INTF1) | (1 << INTF0) | (0 << INTF2);

    #asm("sei") // Enable global interrupts

    while (1)
    {
        separte_digits(); // Update digit separation
        display_digits(); // Display hour, minute, and second
    }
}

```

Discussion:

In the updated code, the stopwatch now tracks hours in addition to minutes and seconds. An hour variable is added and the `seperate_digits()` function to break the hours into two digits for display on the 7-segment. The `display_digits()` function now shows hours, minutes, and seconds.

The Timer1 interrupt was adjusted to increment the hour when minutes reach 60, and the hour resets to 0 after 24, keeping it in a 24-hour format.